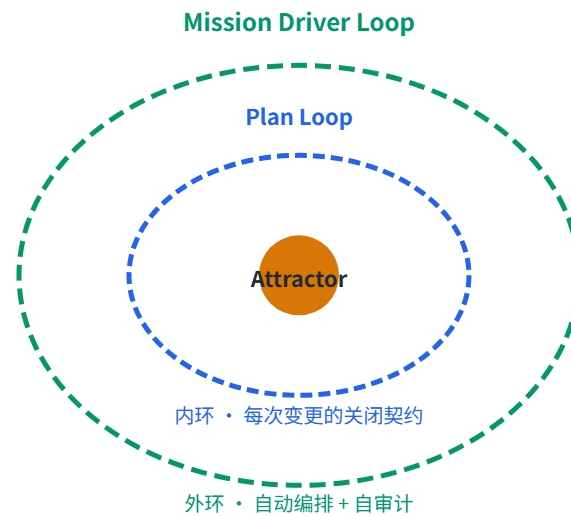


Loop Engineering × Attractor

Attractor Guided Loop 实战—— 中型 ERP 产品的 AI 全自动开发

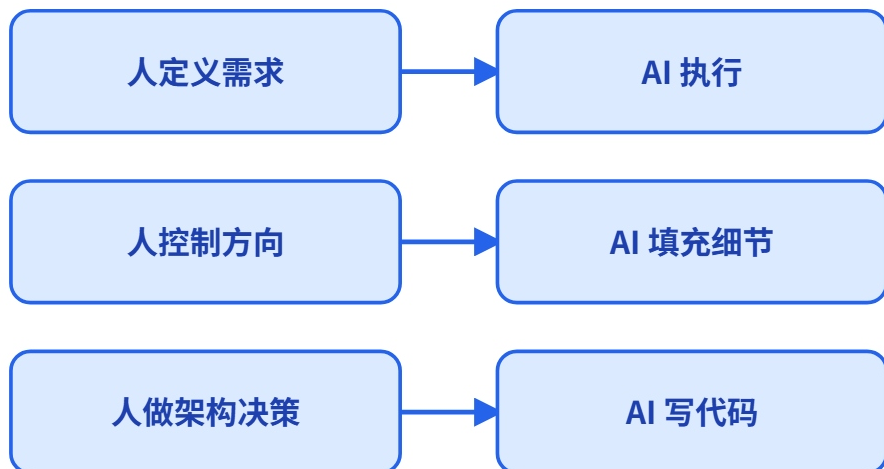


Nop Platform · AGE workflow · 22 天 · 154 模块

AGE = Attractor-Guided Engineering (吸引子引导工程)

变革点：传统软件工程的智能时代转型

✓ 传统信念



🧪 nop-app-erp 的实验

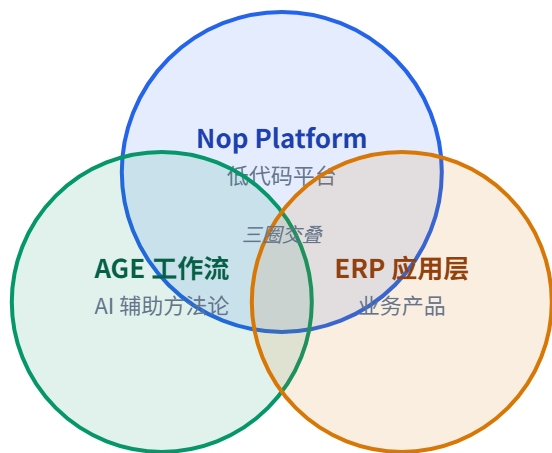
nop-app-erp 是探索 AI 全自动开发边界的实验性项目。核心追问：**如果没有人，AI 还能产出同样的系统吗？**

实验记录：全量抽取 28 天日志，得到 28 条明确人类干预 + 80 次设计决策记录

继续追问：这 28 条干预 + 80 次决策，多大比例可以被另一 AI agent 替代？

初步判断：约 90% 的知识内容属于通用可编码类型，约 10% 涉及方向性/战略性判断

nop-app-erp 是什么



18+1

业务域

154

Reactor 模块

352

自有实体

22

天

一句话定位：基于 Nop 平台的**产品化通用 ERP 参考应用**
(应用层产品，非框架核心)

起点：06-22 上午的真实状态

```
└─ nop-app-erp/
  └─ module-master-data/ [有结构, 实体≈0]
  └─ module-inventory/ [空]
  └─ module-purchase/ [空]
  └─ module-sales/ [空]
  └─ module-finance/ [空]
  └─ module-assets/ [空]
  └─ module-projects/ [空]
  └─ module-manufacturing/ [空]
  └─ module-quality/ [空]
  └─ docs/ [模板占位符]
  └─ pom.xml
```

⚠ 构建未闭环 / 测试数=0 / 竞品调研=空白 / i18n缺失

AGE 模板初始化后的“空骨架”

核心问题



10 域 ORM 有文件但多数实体为 0

只有 master-data 等少数域有占位实体，其余域为空白模板



构建未闭环

无聚合 app，模块间依赖断裂，`mvn clean install` 失败



测试数 = 0

无单元测试，无集成测试，无 E2E 基础设施

从这样一个空骨架到产品级 ERP，需要多久？靠什么？

22 天后：经审计校准的关键规模指标



352+110

自有实体 + 引用桩 ≈ 462 ($\approx 16/\text{天}$)



154

Reactor 模块 (06-23 首闭环 81
→ 全绿 154)



$\sim 2,890$

Java 测试 / 0 failure / 0 error



260+

Playwright E2E spec / 0
regression



24+10

报表 (24 份) + 看板 (10 个)



19

可复用技能 (docs/skills/)



187+2

计划文件 + 指南



9+78

审计记录 9 / 分析文件 78

这些是**经审计校准**的口径——不是“22 天的奇迹”，而是三层控制 Loop 的系统输出产物

从 Prompt → Loop → AGE

“ Boris Cherny (Claude Code)
"我不再写 prompt 了。我设计的 Loop 在 prompt Claude。"

“ Addy Osmani (Google)
"Loop Engineering = 你不再写 prompt，而是设计写 prompt 的系统"

“ Andrew Ng 三层 Loop
Agentic Coding (分钟) → Developer Feedback (小时) → External Feedback (天/周)

隐含前提: "方向已知。" AGE 问: **方向谁定义的? 保证方向本身是对的?**

Harness 层 (反馈控制 Loop)

Agentic Coding
分钟级

Developer Feedback
小时级

External Feedback
天/周级

需要一个方向 →

AGE: 方向先于控制

状态空间

吸引子

轨迹

控制

“ Loop Engineering 让 AI 跑得更快更稳。Attractor 让 Loop 知道往哪里跑。

Harness: Plan + Mission Driver

Attractor: 长期收敛目标

Vibe Coding Loop

🔄 传统 AI 开发：用户说 → AI 写 → 用户改 → AI 再写 → ... ∞ 无退出条件

四大缺失

✖ 无基线

不知道当前状态，无法衡量变更

✖ 无范围

不知道要做什么，边界模糊

✖ 无退出标准

不知道何时算"完成"

✖ 无审计

没有人独立验证"是否正确"

从 In 到 On 的关键：Plan

Plan 不是任务清单，是**关闭契约**：

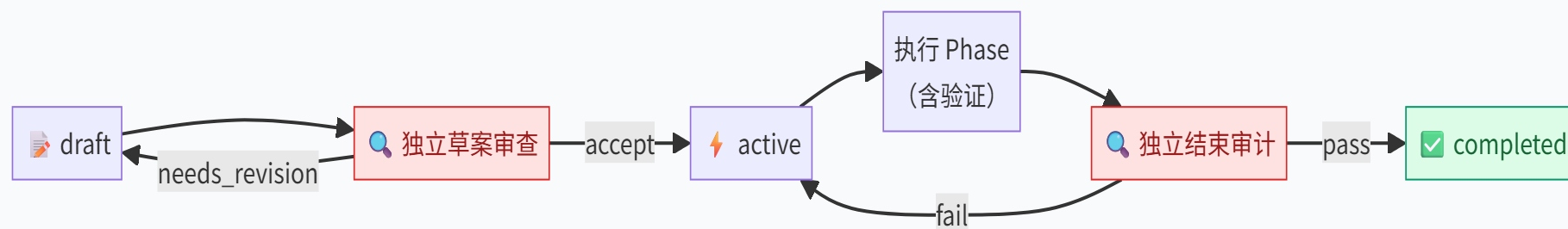
Current Baseline

Goals + Non-Goals

Exit Criteria

Closure Gates

Draft Review Record



🛡️ **生成与验收分离**：审计必须由独立子代理（fresh session）执行，不继承执行上下文

🔗 这套 Plan + 审计 + 验证的联合机制，是 Harness 中的 **Plan Loop**—— 控制单次变更轨迹的执行层

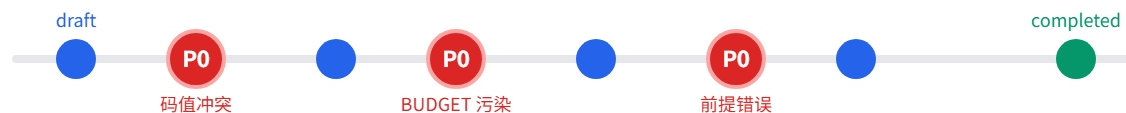
实例：Plan 1100-1 销售定价引擎

draft → 审查 iter1 (“客户维度 Decision 缺失”, needs_revision)
→ 修订 → 审查 iter2 (accept)
→ active → Phase 1-4 (ORM+ SPI + 测试 + 前端)
→ mvn test 全绿 → 结束审计 PASS → completed



该批次 4 个 Plan 拦截 4 个 P0 缺陷—— 码值冲突、BUDGET 污染财务、GIBalance 前提错误、维度歧义。全部在编码前拦截

如果没 Plan Loop：这 4 个缺陷在实施/运行期才暴露，修复成本高 **5-10 倍**



为什么需要更大一层的 Loop

? Plan Loop 只管理"一次变更"

谁决定下一个？谁确保全路线图被执行？谁空闲时触发审计？



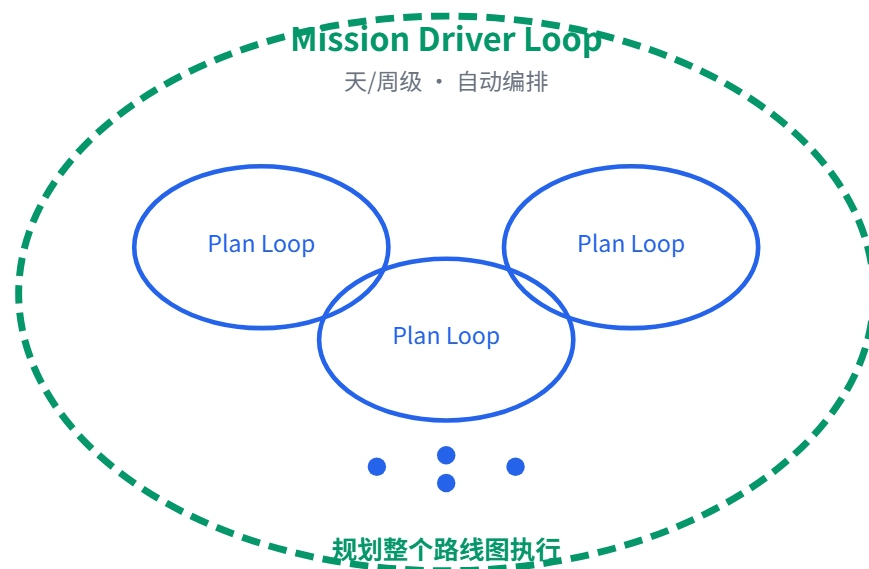
Mission Driver = AGE 框架中 harness 层的自动化实现

在 Plan Loop 之外加一个编排层。完全依赖人 → 手动、不可重复。



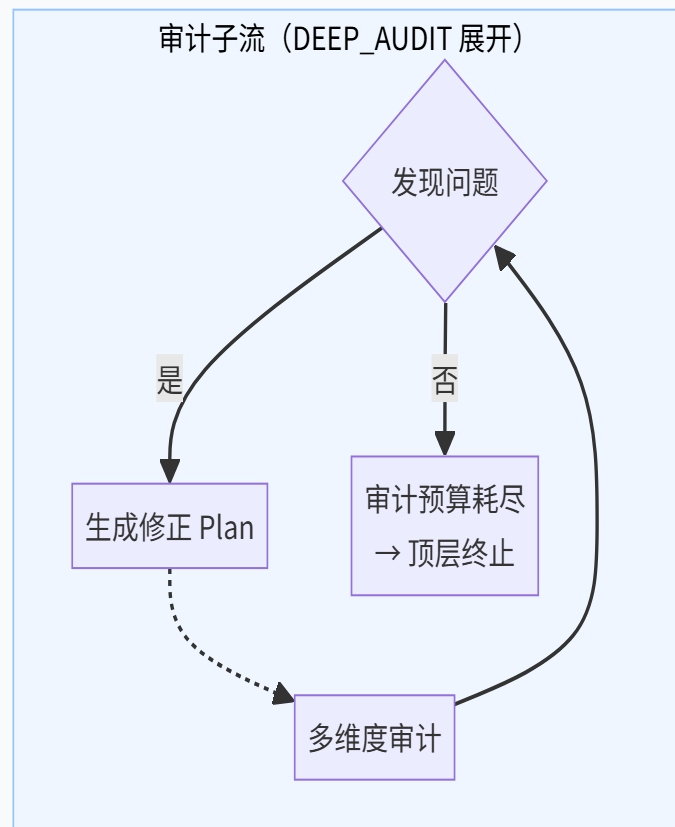
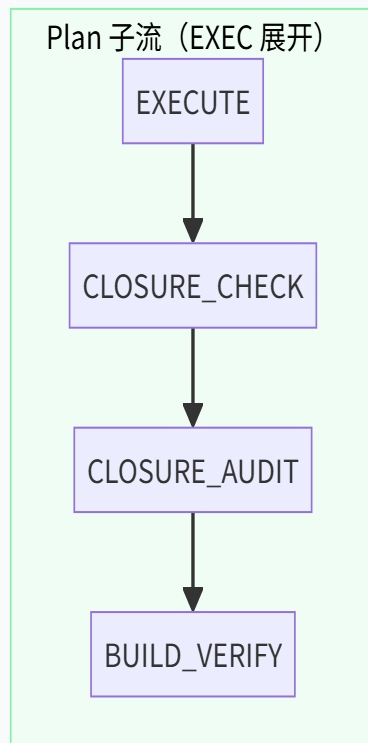
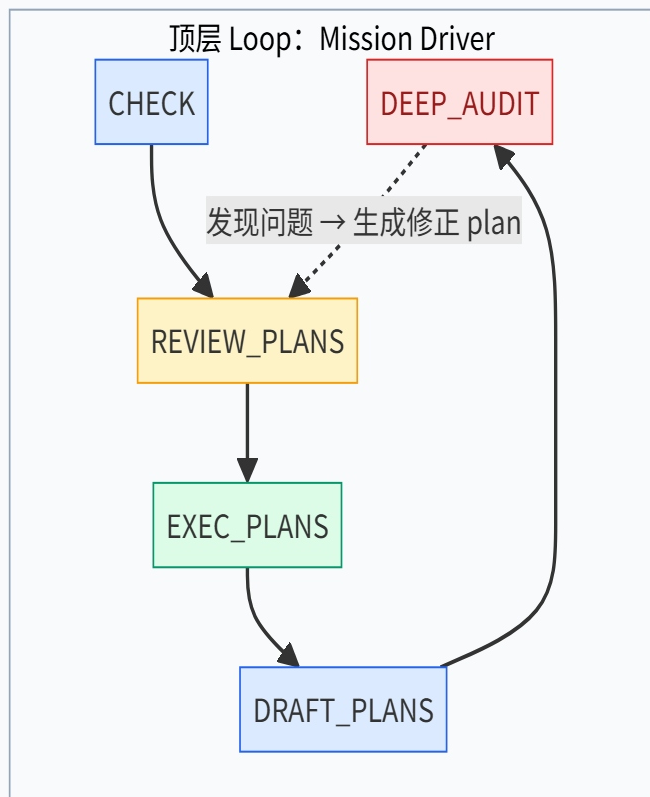
时间尺度：天/周级

一次完整循环处理 1-7 个计划、耗时 4-12 小时。整个路线图的流转需要数天到数周。它自动运行的底气，来自 **attractor 已被定义好**



🔧 Mission Driver 的 5 步闭环

🔄 Vibe Coding 是一条**无限长单一 Loop**（没有退出条件，用户喊停才停）。Mission Driver 的核心贡献：将其分解为**三个嵌套的 Loop**，每个有终止条件、有隔离边界。理解了这个结构，就理解了它为什么能长时间稳定运行。



Mission Driver 的实现架构——通用 Flow DSL 引擎



关键设计一：注入机制

AI step 的 prompt 模板含 `{{planGuide}}`、`{{roadmapPath}}` 等占位符，引擎从 mission.json 配置中读取真实路径并注入。AI 的思考内容不来自 prompt，而来自外部注入的 plan guide、roadmap、context docs。换项目只需改配置，不改 prompt

关键设计二：隔离的顶级步骤

CHECK / REVIEW_PLANS / EXEC_PLANS / DRAFT_PLANS / DEEP_AUDIT 相互不传参数。所有状态通过 Plan 文件 (`> Plan Status:` + `[x]` 复选框) 和 Audit 文件持久化到磁盘。因此可以：单独执行任一步骤、从任意步骤开始向后执行、恢复不依赖内存——扫盘即知"当前在哪"

关键设计三：零 IPC，全文件

没有 IPC、没有 web server。Mission Driver 主进程只做状态机推进，AI 通过文件 I/O 交互。这是 AGE "仓库 = 唯一真相源"原则在工具层的落实

对人介入的支持

DRAFT_PLANS 持续起草新 Plan；REVIEW_PLANS 自动拾取 Status=draft 的 Plan（不区分来源）；人可随时修改 Plan 状态/路线图，下轮循环自动适配

真实运行记录：07-10 一次完整循环

```
08:00 CHECK      mvn 全绿
08:05 REVIEW_PLANS 4 个 draft → all active
09:30 EXEC_PLANS  4 个 Plan 全绿 ✓
16:00 DRAFT_PLANS 路线图全 done
16:05 DEEP_AUDIT  自动启动深度审计
```

```
# 一次循环 ~8h (08:00 → 16:05)
# 处理 4 个计划 • 经历 5 个步骤
# 路线图横跨 22 天 (06-22 → 07-13)
```

5 步自动循环，每步都是 AGE harness 层的具体实现。

注意看时间尺度：从 CHECK(08:00) 到 DEEP_AUDIT(16:05) 约 8 小时，完成了一次完整的 Mission Driver 循环——处理 4 个计划、经历 5 个步骤。而整个路线图（18 域 + 业财 + 扩展域）从 06-22 到 07-13 横跨 22 天，Mission Driver 在这期间完成了数十次这样的循环。

两层 Harness 的关系—— 以及它们不能做的事

Plan Loop

单次变更控制。每次变更从起草到关闭的完整管理

Mission Driver Loop

编排控制。自动遍历路线图，决定"下一个做什么"，空闲时自动审计

但它们不能回答：路线图本身正确吗？"正确的结构"是什么？

07-12 竞争杠杆审计发现的"2 个夸大 + 2 个缺口"—— 不是因为 Harness 没执行好，是因为**方向定义层（Attractor）有过问题**（文档夸大"多公司 ERP"、制造委外曾是空壳）。Harness 执行得越好，方向错了损失越大。



过渡到 Act 4："所以我们需要解决的，不是第三个 Loop—— 而是一个完全不同性质的东西：系统长期收敛的目标结构（Attractor）。"

动力系统语言——为什么需要 Attractor



三层结构：结构层 → 承载层 → 实现层

吸引子不是文档（文档是承载），吸引子不是代码（代码是投影）

混淆 1：吸引子 ≠ 边界

边界回答"什么不能做"，违反即报错。吸引子回答"系统应向哪里长期收敛"，单次偏离不报错但持续偏离让结构腐烂。反例：规定"不准用 JPA"（边界），AI 绕过用 JDBC 写 SQL——没违反边界但破坏了"ORM 模型唯一真相源"这个吸引子。边界抓不住这种偏离。

混淆 2：吸引子 ≠ 更强的护栏

护栏在执行层检查每次动作，吸引子在方向层评估多次动作累积是否靠拢。反例：10 次 PR 的代码规范都通过（护栏），但测试套件已与旧实现深度耦合。护栏视角：每次合规；吸引子视角：方向在漂移。

混淆 3：吸引子 ≠ 控制目标

它是控制的目标因，不在控制之中。没有先定义 attractor，Harness 没有统一含义。反例：团队有完美的 CI/CD + code review + 测试门禁，但没人回答过"系统该向什么架构收敛"。结果：所有控制机制高效地固化了一套错误基线。

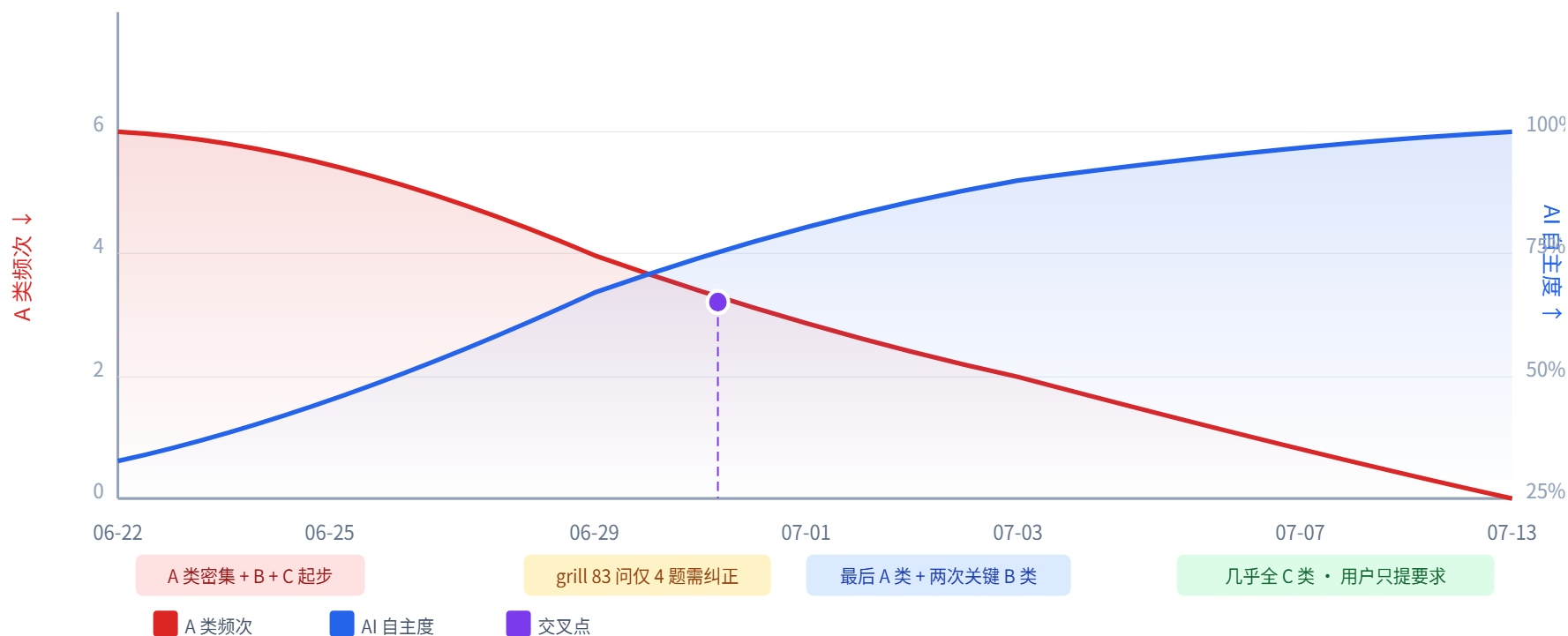
用户其实在做什么—— 定义和演进吸引子

介入类型	频次	用户做了什么	对吸引子的影响
 A 类	9 次 06-22 ~ 07-03	明确指明平台机制 "租户开关是全局配置，不是每个实体手写"	修正吸引子的技术基底
 B 类	9 次	指明工程原则方向 "过账方法应挂聚合根 BizModel"	修正吸引子的结构原则
 C 类	绝大多数 07-04 后为主	只让 AI 自查对比 "对照 erp-survey 检查功能覆盖"	在吸引子内让 AI 自行扩张



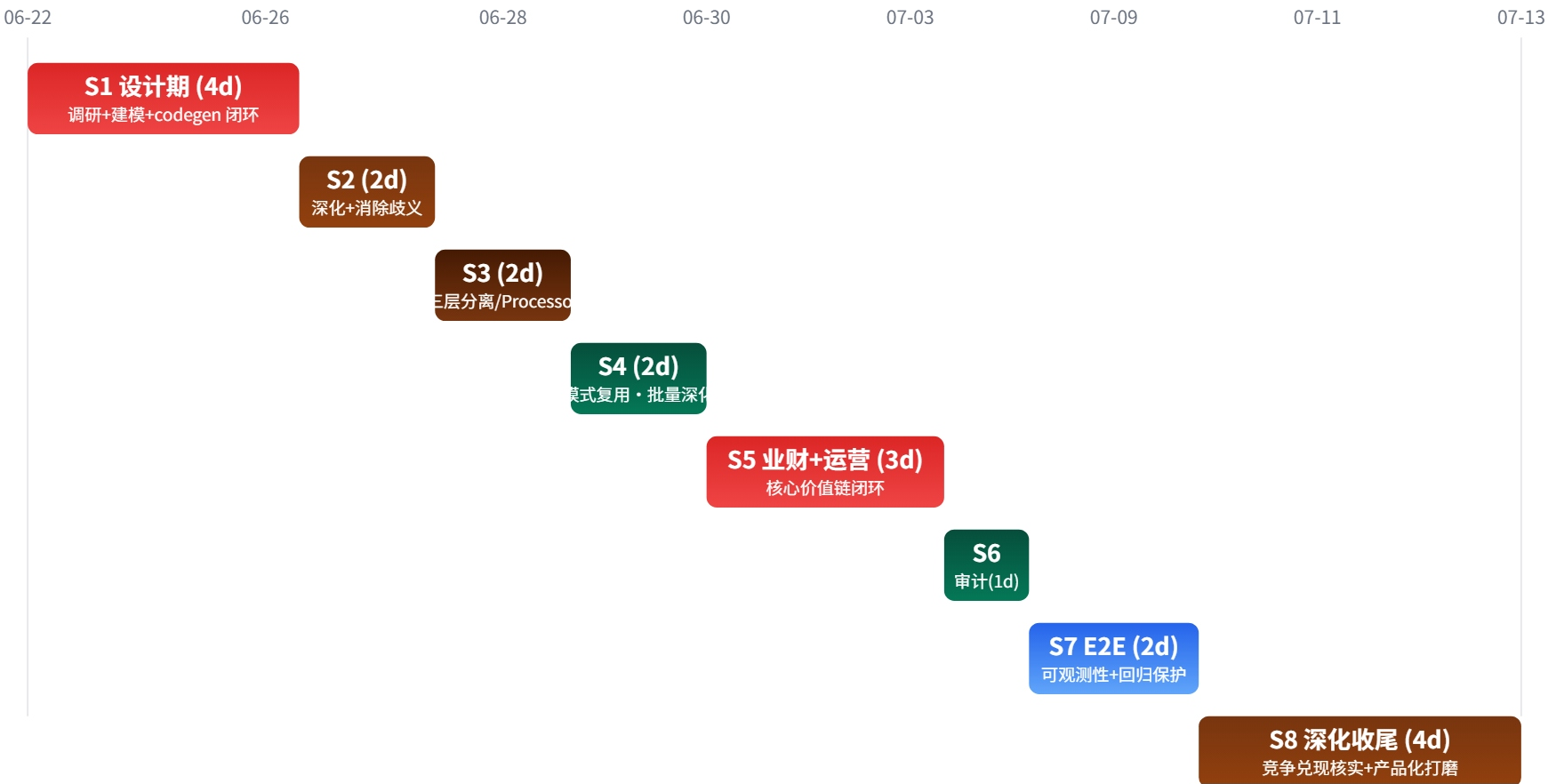
关键洞察：用户后期介入归零不是因为 AI 学会了写代码，而是因为**吸引子已经定义好了**。方向对了，AI 能自动扩张。

知识转移曲线



💡 "用户不是在写代码，而是在**转移平台知识**。一旦知识转移完成，他的介入频率自动归零。"

8 阶段演进 (06-22 → 07-13 / 22 天)



节奏特征：两头重（前 4 天调研+建模+codegen 闭环，后 4 天业务深化+竞争兑现核实）

六种自驱动机制

 1. 计划-审计闭环

187 计划，强制双审计

 2. 路线图驱动

3 子路线图逐项推进

 3. 审计-整改驱动

9 轮审计

 4. 测试回归防护

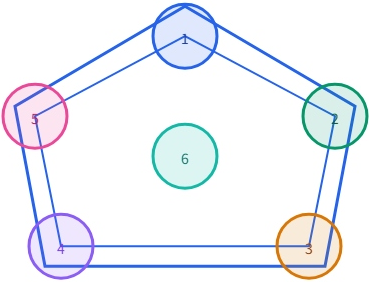
~2890 单元 + 260 E2E

 5. 竞品对标驱动

24 项目作完整性参照系

 6. 文档一致性驱动

ORM XML 唯一真相源



对 AI 自主开发研究的含义

提供了一个**可公开审计的实证案例**：在限定条件下（通用产品、无真实客户约束），AI 可以自主完成从 ORM 模型到 E2E 测试的全栈开发。之前的讨论停留在"AI 能不能写代码"—— 这里的答案是**"能，而且能完成需求分析、设计决策和架构选择"**

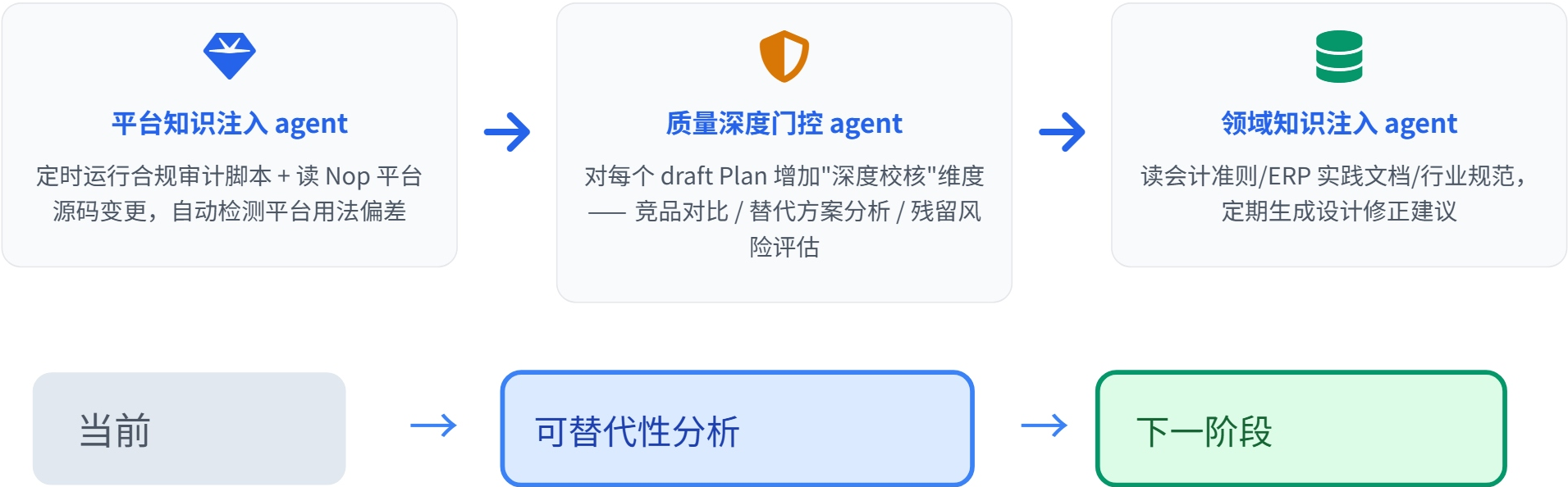
≡ "全自动 AI 开发"的适用条件：



这些条件恰好定义了一个"产品化通用软件"的范畴

从 28 条干预到全自动的最后一公里

需要构建三个基础设施：



三个 Takeaway

Takeaway 1

把"变更"从对话变成契约

Plan Loop 给每次变更一个关闭契约
(draft→审查→执行→审计
→completed)

这是 Human On The Loop 的前提——
AI 自主执行完整一轮，不等人

Takeaway 2

把"编排"从记忆变成自动

Mission Driver 自动遍历路线图、拾取
Plan、审查执行、空闲时审计

Plan 够了吗 → 不够 → Mission Driver
来编排—— 这是从 In 到 On 的关键跨越

Takeaway 3

把"方向"从个人变成文件

Attractor 承载于 owner docs，可版
本、可审计、可继承

吸引子不能外包给 AI—— 定义方向是人的
工作

实战方案—— 三种项目模式



模式 A：手工驱动

Plan 机制：每次变更加 Plan + 独立审计，人驱动路线图

手工执行 Plan 生命周期，零工具依赖

人工驱动



模式 B：路线图驱动

Roadmap 已就绪（AI 生成或手工调整）
Mission Driver 根据 Roadmap 全自主运行

自动驱动



模式 C：愿景驱动

用户描述简短目标，Mission Driver 自动运行技能生成 Roadmap
然后全自主执行—— 对用户表现为许愿式开发

许愿式开发

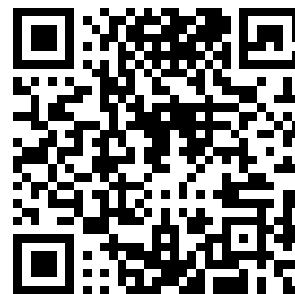
Q&A

问题与讨论

微信公众号



交流群



`github.com/canonical-entropy/nop-app-erp`

`github.com/canonical-entropy/attractor-guided-engineering-template`

谢谢!